CSC3002F assignment – Networks 1

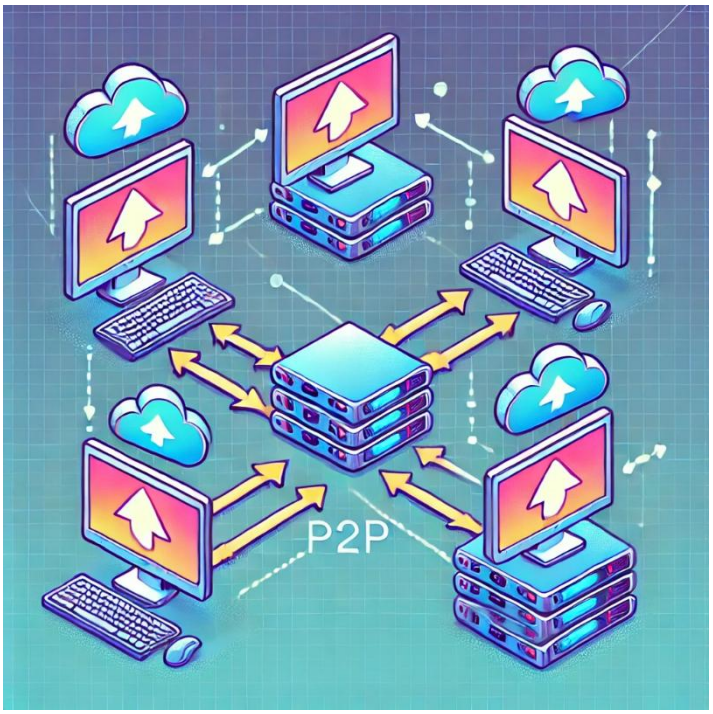
March 2025

BY:

Sarah-Jane Marais MRSSAR011

Daniel Schlesinger SCHDAN066

Matome Mbowene MBWMAT002



Contents

Introduction 3
Protocol specification 3
Message formats and types 3
The control message format is as follows: 3
The command message format is as follows: 4
The data message format is as follows: 4
States of hosts 4
Sequence diagram 5
Representing the process of downloading file: fileName 5
Representing the continuous messages from Seeder to tracker 5
Application features 6
Tracker 6
Seeder 6
Leecher 6
Extra features and classes 6
Feature images 7
Conclusion 11

Introduction

This report presents the design and implementation of an application layer Peer-to-Peer (P2P) File Sharing System. The system enables users to share and download files in parallel in a decentralized network, employing a Tracker, Seeders, and Leechers. This system was created in python using TCP and UDP socket infrastructures. The communication is based on TCP and UDP protocols. TCP sockets are used for the transfer of the file to ensure reliable data transfer. UDP sockets are used for the available Seeders requests and “hellos” to update the directory of the Tracker, in an efficient but less reliable manner. The inclusion of these aspects, coding logic and parallel computing concepts, together make an application layer p-2-p network on the local device.

**Failed to get the code to run properly and couldn’t locate the issue even after testing was done

Protocol specification

The system follows a custom simplified message protocol to facilitate communication between Trackers, Seeders, and Leechers. The message protocol was simplified to what this system model needed. The message header was kept consistent for all messages and the body is the defining characteristics for the types of the methods.

Message formats and types

Messages exchanged in the system use a standardized format:

- **Header (5 bytes):** Contains the message ID (1 byte) and body length (4 bytes).
- **Body:** Contains the actual message content.

The control message format is as follows:

Header: [msgID (1 byte)] [bodyLength (4 bytes)]

Body: [message content]

The control messages that are part of the system are:

- The acknowledgements of receipts of messages
  (msgID = 0x01) [message body = acknowledgement],
- the constant “hello” messages from the Seeders to inform the Tracker they are still active and part of the system
  (msgID = 0x01) [message body = fileName, timestamp],
- and the informing of requested data not found/available
  (msgID = 0x10) [message body = data not found acknowledgement].

The command message format is as follows:

Header: [msgID (1 byte)] [bodyLength (4 bytes)]

Body: [message content]

The command messages that are part of the system are:

- The request for available Seeders by the Leecher to the Tracker,
  (msgID = 0x06) [message body = fileName
- and the request for a file chunk by the Leecher to the Seeder.
  (msgID = 0x07) [message body = fileName, begin, end]

The data message format is as follows:

Header: [msgID (1 byte)] [bodyLength (4 bytes)]

Body: [message content]

The data messages that are part of the system are:

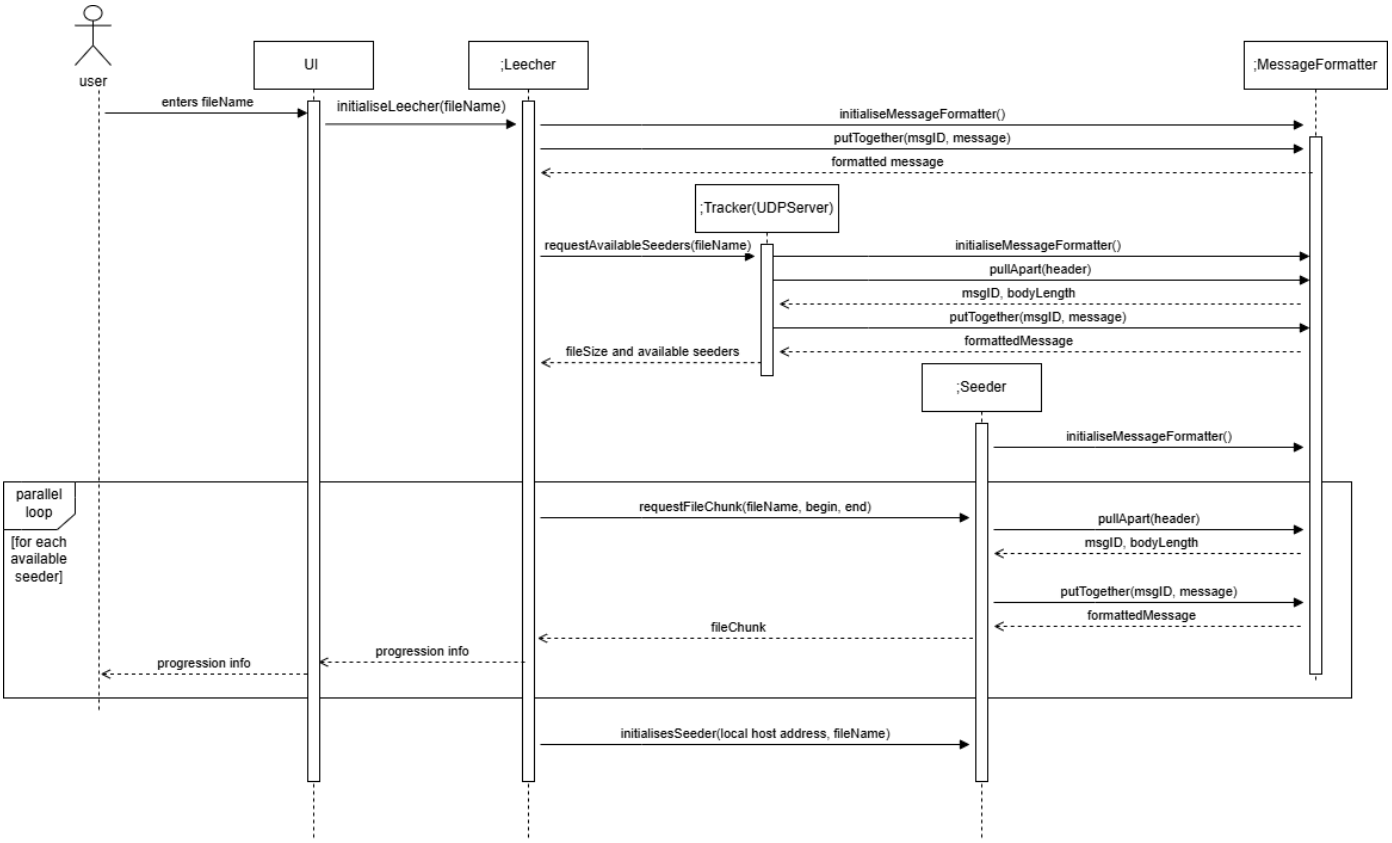
- The file chunk being sent from the Seeder to the Leecher,
  (msgID = 0x05)[message body = requested data: fileChunk]
- and the available Seeders list sent from the Tracker to the Leecher
  (msgID = 0x05) [message body = requested data: fileSize, available Seeders List]

States of hosts

In this system we have a clear method of determining the 'availability' of the host servers/Seeders. The Seeders continuously let the Tracker know they are still 'available' through messages sent every 15 seconds(not fully implemented but code is there and commented out). If the timestamp of the Seeder's record in the Tracker's directory is outdated, they are removed and no longer 'available'. The no longer available Seeders therefore won't be given to the Leechers and therefore won't be sent messages requesting the file chunks. If the tracker responds with no Seeders for the file then none are available and the file cannot be accessed.

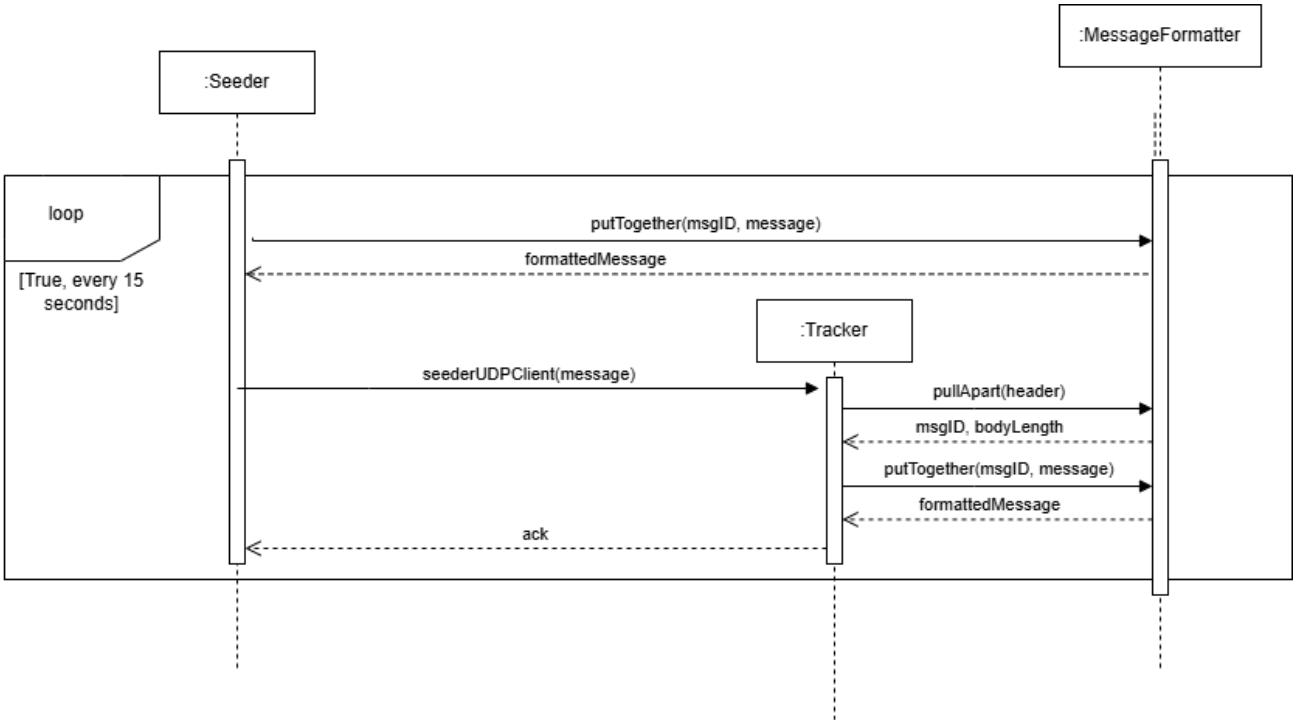
Sequence diagram

Representing the process of downloading file: fileName



Representing the continuous messages from Seeder to tracker

(code commented out as had issues)



Application features

The structure of the system is made up of three main parts and some additional features.

Tracker

The tracker is a UDP server-side socket that receives two types of messages:

- requests for available Seeders (0x06)
- and acknowledgements from the Seeders (0x01) every 15seconds to inform the Tracker that they are still active.

It responds to Leecher requests for available Seeders using the trackerUDPServer() and formulateResponse() methods. It responds with a list of available Seeder addresses and the file size that is used by the Leecher for its file chunk requests. The tracker maintains an up-to-date list of all the Seeders in the local p-2-p network and the files that they are a Seeder/server for. The tracker keeps the directory up to date by looping through the elements in the directory and removing any that have a timestamp earlier than 15 seconds before the current time (dateTime and time classes are used for this.) The method updateDirectory() is used for this. It is nonfunctional at the moment but commented in the tracker class.

Seeder

The Seeder is a TCP server-side socket that hosts the files on the p-2-p network. The Seeder receives messages from the Leecher requesting file chunks for a file. The chunk is taken out of the downloaded file, fileName, from the 'begin' index to the 'end' index which are given as parameters to the seederTCPServer() method. The chunk is then sent to the requesting Leecher. The Seeder also continuously sends messages to the Tracker to keep the directory up to date. It does this using the dateTime class and a while loop that runs while TRUE sleeping for 15seconds at a time before calling the SeederUDPClient() method.

Leecher

The Leecher class is a UDP and a TCP client. It sends a request to the Tracker across the UDP socket and method leecherUDpClient() to get available Seeders. The response of available Seeders is used to calculate the number of chunks the file should be divided into and further get the index for the begin and the end. The Leecher then uses the available Seeders list to then send out requests to the available Seeders for chunks across the TCP socket connection and using the leecherTCPClient().This is done in parallel. The chunks are then retrieved from the socket connection and put back together.

Extra features and classes

- **MessageFormatter:** Handles message formatting and encoding and decoding with two methods. The pullApart() method takes in the header of the message as a parameter and returns the msgID and the bodyLength. The putTogether() method takes in the msgID and message and calculates the bodyLength. The msgID, bodyLength and message are encoded and sent back.

- **Parallel Downloads:** The Leechers send requests for file chunks to the available Seeders in parallel. The file chunks are then sent back to the Leecher and ordered correctly by future objects waiting on the thread that was made before.
- **UI:** The simple UI for the system has an input function for a file name to be downloaded and progress bar. Commented out as non-functional: The progress bar updates by the percent as the chunks come in using a separate thread and a shred variable between the threads active.

Feature images

Screenshots of the application revealing its features.

- **Tracker UDP server-side socket**

```
def trackerUDPServer(self):
    self.serverSocket = socket(AF_INET, SOCK_DGRAM)
    self.serverSocket.bind(('127.0.0.1', self.port))
    print("The server is ready to receive")
    while True:
        message, addr = self.serverSocket.recvfrom(2048)
        print("[MESSAGE]", message)
        msgID, bodyLength = self.msgF.pullApart(message[:5])
        self.formulateResponse(message[5:], addr, msgID, )
```

- **Seeder TCP server-side socket**

```
def seederTCPServer(self):

    serverSocket = socket(AF_INET, SOCK_STREAM)
    serverSocket.bind(('127.0.0.1', self.port))
    serverSocket.listen(1)

    while True:

        connectionSocket, addr = serverSocket.accept()
        header = connectionSocket.recv(5)
        msgID, bodyLength = self.msgF.pullApart(header)
        message = connectionSocket.recv(bodyLength)
        message = message.decode()
        fileName, begin, end = message.split(",")
        print(f"Received request for {fileName} from {addr}")
        begin = int(begin)
        end = int(end)


        # respond with the asked for file chunk
        fileChunk = self.getFileChunk(fileName, begin, end)
        fileChunk = fileChunk.decode()
        message = self.msgF.putTogether(0x05, fileChunk)
        connectionSocket.send(message)
```

```
def getFileChunk(self, fileName, begin, end):
    with open(fileName, 'rb') as file:
            file.seek(begin)
            numBytes = end - begin + 1
            extractedBytes = file.read(numBytes)
    return extractedBytes
    # retrieve the chunk required and send back to be encoded and sent to client

# method to update the tracker with the files the seeder has available by calling the SeederUDPClient method every 15 seconds
# def updateUDPDirectory(self):
#     while True:
#         self.SeederUDPClient()
#         time.sleep(15)

# method to send a message to the tracker with the files the seeder has available
def SeederUDPClient(self, fileName, fileSize):
    timeStamp = datetime.now()
    body = fileName + ","+str(fileSize) +","+ str(timeStamp)
    message = self.msgF.putTogether(0x01,body)
    try:
        udpSocket = socket(AF_INET, SOCK_DGRAM)
        udpSocket.sendto(message, ('127.0.0.1', 3535))
        message, clientAdd = udpSocket.recvfrom(1024)
        address = message[5:].decode()
        address = ast.literal_eval(address)
        self.port = address[1]

        print(f"[RECEIVED SEEDER] {str(self.port)}")

    except Exception as e:
        print(f"An error occurred in leecherUDPClient: {e}")
```

- **Leecher UDP client-side socket**

```
def leecherUDPServer(self, fileName):
    udpSocket = socket(AF_INET, SOCK_DGRAM)
    udpSocket.bind(('127.0.0.1', 0))  # Bind to available port

    self.fileName = fileName

    try:
        # Send request to tracker
        message = self.msgF.putTogether(0x06, fileName)
        udpSocket.sendto(message, ('127.0.0.1', 3535))
        print(f"Request for {fileName} sent to tracker")

        # Wait for response
        print("Waiting")
        message, addr = udpSocket.recvfrom(1024)
        print(f"[RECEIVED] {message}")

        # Process response
        msgID, bodyLength = self.msgF.pullApart(message[:5])
        print("MSGGGGGG   " + str(msgID))
        if msgID == 0x05:
            body = message[5:].decode()
            strFileSize, availableSeeders = body.split(",", 1)
            fileSize = int(strFileSize)
            print(f"File size: {fileSize}, Available seeders: {availableSeeders}")

            # Start downloading file chunks
            self.parallelTCPRequest(fileSize, availableSeeders, self.fileName)
            return True
        else:
            print(f"No seeders found with the file: {self.fileName}")
            return False


    finally:
        udpSocket.close()
```

- **Leecher TCP client-side socket with parallel requesting**

```
def parallelTCPRequest(self,fileSize, availableSeeders, fileName):
    # available seeders is a list of the seeders and numSeeders the number of seeders available
    availableSeeders = ast.literal_eval(availableSeeders)

    numSeeders = len(availableSeeders)
    chunkSize = math.ceil(fileSize/numSeeders)  # fileSize will be stored and worked with as bytes
    # chunkCount will be used to control the parallel requests
    chunkCount = 0
    threads = []
    resultsDict = {}

    for i in range(numSeeders):
        begin = chunkCount*chunkSize
        end = min((begin + chunkSize - 1), fileSize)
        chunkCount = chunkCount + 1

        message = fileName + "," + str(begin) + "," + str(end)
        formattedMessage = self.msgF.putTogether(0x07, message)

        print("here look" + str(availableSeeders[i]))

        def download_chunk(seederAddr, msg, chunkIndex, resultsDict):
            chunkData = self.fetchChunk(seederAddr, msg, numSeeders)
            if chunkData:
                with self.lock:
                    resultsDict[chunkIndex] = chunkData
                    print("CHUNK INDEX"+ str(chunkIndex))

        thread = threading.Thread(
            target=download_chunk,
            args=(availableSeeders[i], formattedMessage, i, resultsDict)
        )
        threads.append(thread)
        thread.start()
```

```
    # Waits for all threads to complete
    for thread in threads:
        thread.join()

    # chunks are then reordered
    orderedChunks = [resultsDict[i] for i in sorted(resultsDict.keys()) if i in resultsDict]

    # code to download file to device
    with open("output/"+fileName, "wb") as f:
        for dataChunk in orderedChunks:
            f.write(dataChunk)
    print("Download complete")

    # s = Seeder(self.localPort, fileName)
    # thread = threading.Thread(target=s.seederTCPServer)
    # thread.start()
    # thread2 = threading.Thread(target=s.updateUDPDirectory)
    # thread2.start()
```

```python
#connnects to a seeder via TCP, then requests a file chunk and receives the requested data
def leecherTCPClient(self, serverAddress, message):
    print(serverAddress)
    try:
        clientSocket = socket(AF_INET, SOCK_STREAM)
        clientSocket.connect(serverAddress)
        clientSocket.send(message)

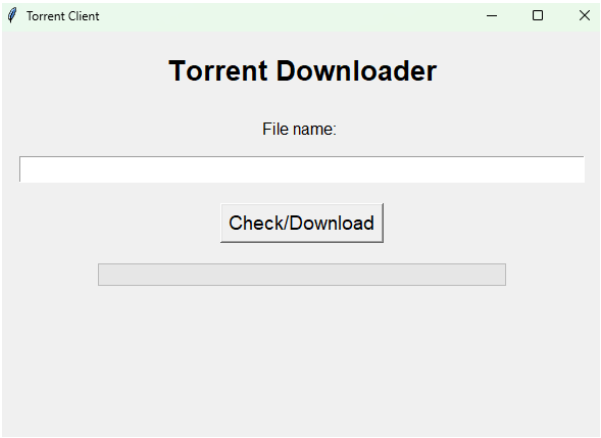
        header = clientSocket.recv(5)
        msgID, bodyLength = self.msgF.pullApart(header)
        fileChunk = clientSocket.recv(bodyLength)
        localIP, self.localPort = clientSocket.getsockname()
        return fileChunk
    except Exception as e:
        print(f"An error occurred in leecherTCPClient: {e}")

# fetching the chunk that also keeps track of how many chunks downloaded and updates the progresss bar
def fetchChunk(self, seederAddress, message, totalChunks):
    fileChunk = self.leecherTCPClient(seederAddress, message)

    #with self.lock:
   #     self.chunksDownloaded += 1
  #      self.updateProgress(self.chunksDownloaded, totalChunks)

    return fileChunk
```

- **UI**



- **messageFormatter class**

```python
# Message formatter
# Because we have a standard message format we will be required to run through these methods many times. We
# have placed the formatting and deformatting of the header together in this class.
# FORMAT: header(10bytes)--> msgID(4bytes), msgType(2bytes), bodyLength(4bytes) body(bodyLength bytes)

class MessageFormatter:

    @staticmethod
    # takes the msgID, msgType, and the message and puts it into the standard message format. Returns the whole encoded message
    def putTogether(msgID, message):
        encodedMsgID = msgID.to_bytes(1, byteorder="big")
        encodedMessage = message.encode()
        bodyLength = len(encodedMessage)
        encodedBodyLength = bodyLength.to_bytes(4, byteorder='big')
        result = encodedMsgID + encodedBodyLength + encodedMessage
        return result

    @staticmethod
    # pulls apart and decodes the header given. returns the msgID, msgType and the bodyLength.
    def pullApart(header):
        msgID = int.from_bytes(header[:1], byteorder='big')
        bodyLength = int.from_bytes(header[1:], byteorder='big')
        return msgID, bodyLength
```

Conclusion

This system demonstrates decentralized p-2-p file sharing using Python UDP and TCP socket programming. The parallel implementation of the file chunk requesting and dividing the file into chunks makes for an efficient network system. Future improvements could include improved file integrity measures such as hashing. The system must be put through debugging further to locate the error and fix the running of the system.